

01 / The problem

A Berlin visitor wants to choose a restaurant without juggling many tabs. The application provides real search and practical, evidence-labelled planning information.

Success: find the intended restaurant, inspect fresh details, understand uncertainty and save a place for later. Cloud architecture and reproducibility are the assessment focus.

02 / The experience

Search → confirm restaurant → choose visit time → Tonight at a Glance → save or search again.

The interface uses cream, cherry red, large serif headings and rounded cards. It includes loading, empty, missing-key and provider-error states. No fabricated restaurants appear in the product. Visual browser verification is pending.

03 / Portable implementation

FastAPI serves the HTML/CSS/JavaScript and JSON API from one process. Docker packages that same application. Environment variables configure external services.

Google, weather and AI calls sit in separate adapters. A provider can change without replacing the rest of the application. [FastAPI documentation](#)

04 / Live data and limits

Google Places Text Search and Details return real restaurant data with field masks and attribution. Saved browser storage contains only place IDs. Editorial summaries are shown as supplied, when available.

OpenAI receives only generic time-heuristic context. It does not summarize Google reviews. Weather is current, not a forecast for the selected visit. Places policies

05 / AWS architecture

CloudFront HTTPS + WAF → ALB → Fargate tasks in two availability zones. ECR stores images; Secrets Manager holds API keys; CloudWatch provides logs and an alarm.

The origin connection is HTTP in this draft. A custom domain and origin certificate would complete end-to-end TLS. This limitation must remain explicit.

06 / Infrastructure as code

Terraform creates the VPC, routes, two subnets, security groups, ECR, IAM execution role, secrets, ALB, ECS, scaling, CloudFront, WAF and error alarm.

Bootstrap runs with application tasks disabled. Upload secret values and an immutable image, then enable the service. Provider validation and an AWS plan are pending.

Terraform documentation

07 / Security and reliability

Non-root Docker user, ignored private configuration, backend-only keys, HTTPS provider calls, length/ID validation and safe provider errors.

Two baseline tasks, target health checks, deployment rollback and CPU autoscaling up to four tasks. WAF limits public requests. These are configured capabilities, not measured availability results.

08 / Tests and evidence

Passed: Python syntax, JavaScript syntax, Terraform format/parser checks and three time-zone/scoring unit tests.

All 12 API, adapter and scoring tests passed after the user installed dependencies. The local server started successfully. Docker, live API and deployment verification remain pending.

09 / Reflection and alternatives

App Runner's new-customer closure changed the hosting choice. Standard ECS offers explicit infrastructure; Express Mode could simplify a later deployment.

The single container reduces migration work. The heuristic is useful only as labelled generic guidance. Omitting login/database reduces scope but limits saved-list convenience. Highest priority is completing real execution checks.

10 / Completion checklist

Add private keys → install dependencies → pass API tests → inspect desktop/mobile UI → build and run Docker → review IAM and cost → validate/plan/apply Terraform → test HTTPS URL.

Replace draft labels only after evidence exists. Add real screenshots, student reflection and tutor feedback where actually received. Final package includes phase documents, a two-page abstract, final documentation and source.